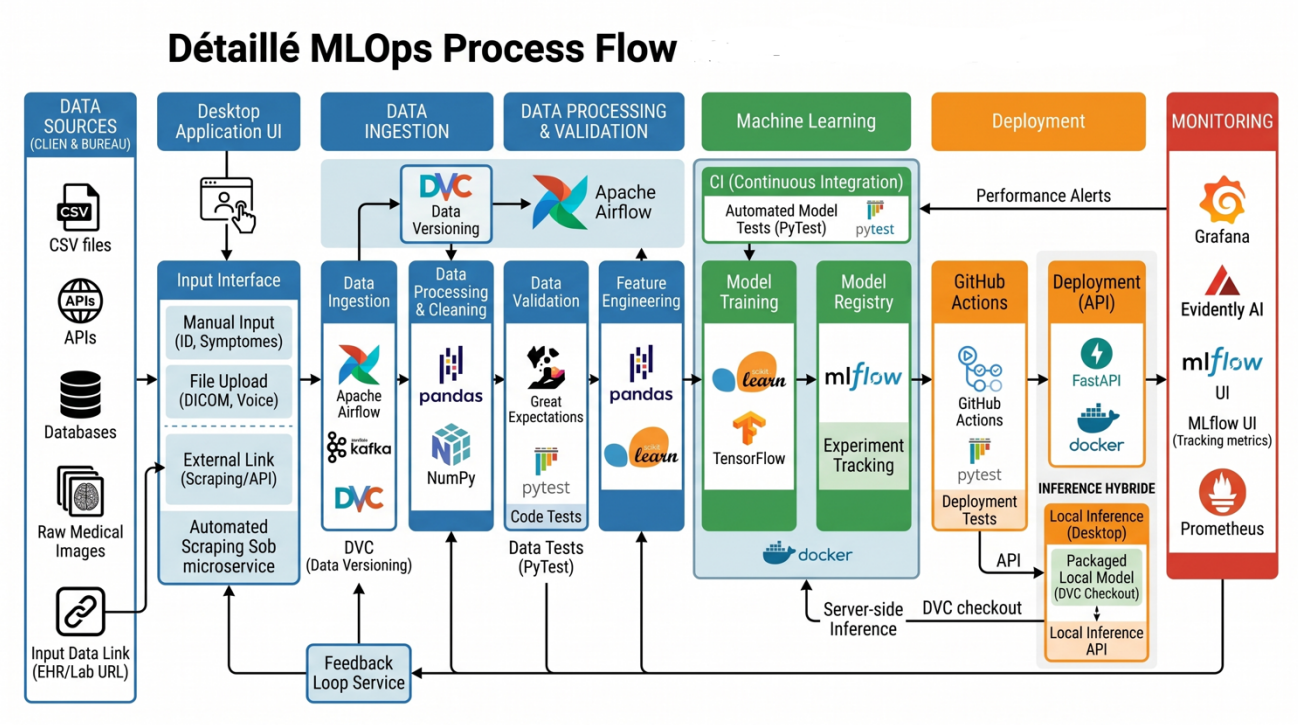
Filière : CI 2 Informatique (IA & BI-DS, Dev, Cyber)
Projet : Machine Learning

1. OBJECTIF

Réalisation d'un projet MLOps (Machine Learning Operations) vise à automatiser et à fiabiliser l'ensemble du cycle de vie d'un modèle de Machine Learning, de sa conception à sa surveillance en production.

Voici les grandes étapes d'un projet MLOps :



2. DEMARCHE :

Phase 1 : Conception et Cadrage

C'est l'étape cruciale pour définir les bases du projet.

- Définition du problème métier : Quel problème essayons-nous de résoudre ? S'agit-il d'une classification, d'une régression, d'une recommandation ?
- Définition des métriques de succès : Comment saurons-nous si le projet est réussi ? Il faut définir des métriques métiers (KPI - ex: augmentation du taux de conversion de 5%) et des métriques ML (ex: Précision, Rappel, F1-Score, RMSE).
- Évaluation de la faisabilité : Disposons-nous de suffisamment de données de qualité ? Avons-nous les compétences techniques et l'infrastructure nécessaires ?

Phase 2 : Collecte et Gestion des Données (Data Engineering)

Cette phase consiste à préparer le carburant du modèle.

- Ingestion des données : Collecter des données à partir de diverses sources (bases de données, APIs, fichiers CSV, logs).
- Exploration et validation des données (EDA) : Analyser les données pour comprendre leur structure, leurs corrélations et identifier les problèmes potentiels (profilage, valeurs aberrantes, biais).
- Nettoyage et prétraitement : Gérer les valeurs manquantes, supprimer les doublons, corriger les erreurs, reformater les attributs.
- Ingénierie des caractéristiques (Feature Engineering) : Transformer les données brutes en caractéristiques (features) pertinentes pour le modèle.
- Versionnage des données : Crucial pour la reproductibilité. Utiliser des outils comme DVC (Data Version Control) pour suivre les versions des datasets.
- Feature Store (Optionnel mais recommandé) : Stocker et gérer les caractéristiques de manière centralisée pour les réutiliser dans différents modèles et garantir la cohérence entre l'entraînement et l'inférence. Outil populaire : Feast.

Phase 3 : Développement du Modèle (Data Science & Expérimentation)

C'est la phase de recherche et de création du modèle.

- Expérimentation : Tester différents algorithmes (Scikit-learn, TensorFlow, PyTorch, XGBoost) et architectures.
- Ajustement des hyperparamètres (Tuning) : Optimiser les paramètres de configuration du modèle pour améliorer ses performances.
- Tracking d'expériences : Suivre et comparer les métriques, les paramètres et les artefacts (modèles) de chaque run. Outil standard : MLflow Tracking.

Phase 4 : Automatisation et Orchestration du Pipeline (ML Engineering)

Cette phase transforme la recherche en un système industriel reproductible.

- Création de pipelines : Transformer les étapes de prétraitement et d'entraînement en scripts robustes et enchaîner ces étapes dans un pipeline automatisé.
- Orchestration : Planifier et gérer l'exécution du pipeline (Ingestion -> Prétraitement -> Entraînement -> Évaluation). Outil le plus populaire : Apache Airflow.

Phase 5 : Validation et Qualité (Tests & Model Registry)

Garantir la fiabilité du modèle avant de le mettre en production.

- Tests unitaires et d'intégration : Tester le code de prétraitement et d'entraînement avec Pytest.
- Validation des données : Vérifier que les données d'entraînement et de production correspondent au schéma attendu avec des outils comme Great Expectations.
- Registre de modèles : Enregistrer le meilleur modèle entraîné dans un registre centralisé pour gérer son cycle de vie (Staging, Production). Outil : MLflow Model Registry.
- Conteneurisation : Empaqueter le modèle et ses dépendances dans une image Docker pour garantir la portabilité.

Phase 6 : Déploiement et Serving (Opérations)

Mettre le modèle à disposition de l'application finale.

- CI/CD pour le ML : Automatiser l'ensemble du workflow (Tests -> Entraînement -> Déploiement) à l'aide d'outils comme GitHub Actions ou GitLab CI. C'est le "Continuous Training" (CT).
- Model Serving (Inférence) : Exposer le modèle via une API (REST temps réel ou Batch). Outils : FastAPI, Flask, BentoML, Seldon Core, TorchServe.
- Infrastructure : Déployer sur une infrastructure scalable, souvent en utilisant Kubernetes.

Phase 7 : Monitoring et Observabilité (Production)

Surveiller la santé et les performances du système en temps réel.

- Métriques techniques : Surveiller la latence, le débit, les erreurs API avec Prometheus et Grafana.
- Monitoring ML (Drift/Performance) : Détecter la dérive des données (Data Drift) ou la dégradation de la performance du modèle (Concept Drift) avec des outils comme Evidently AI ou MLflow UI.

Phase 8 : Gouvernance et Rétroaction (Amélioration Continue)

Assurer la conformité et utiliser les retours pour améliorer le système.

- Gouvernance : Assurer la conformité réglementaire, l'éthique et la traçabilité de chaque version du modèle.
- Boucle de rétroaction : Utiliser les données du monitoring pour déclencher automatiquement ou manuellement le réentraînement du modèle avec de nouvelles données.

3. RENDU DU PROJET

- Document électronique (rapport au moins 20 pages) à rendre via le formulaire.
- Contenu du rapport :
  - Page de Garde
  - Introduction
    - Contexte du projet.
    - Objectifs business et métriques de succès.
    - Motivation pour une approche MLOps.
  - Présentation des Bases de Données (Datasets)
    - Origine et description des données.
    - Exploratory Data Analysis (EDA) et problèmes identifiés.
    - Versionnage des données avec DVC.
  - Présentation des Algorithmes et du Pipeline Technique
    - Architecture MLOps globale.
    - Algorithmes : Modèles testés, choix des hyperparamètres.

- Pipeline Technique :
  - Orchestration des DAGs avec Apache Airflow.
  - Data Validation avec Great Expectations.
  - Amélioration du code : Type hints, docstrings (Google Style), PEP 8.
  - Pipeline de qualité automatique pre-commit (Black, iSort, Flake8, MyPy).
  - Démarche TDD adaptée au ML : Écriture de tests unitaires ciblés avec Pytest pour valider chaque composant (Preprocessing, Fractionnement, Ingénierie).
  - Identification des bugs ML classiques (Data Leakage, Split non stratifié, Mauvaise colonne cible).
  - Distinguer un bug logiciel d'un bug ML.
- Tracking & Registry :
  - Suivi d'expériences avec MLflow Tracking.
  - Cycle de vie dans MLflow Model Registry.

- CI/CD, Déploiement et Monitoring

  - CI/CD Pipeline avec GitHub Actions.
  - Deployment Tests automatisés.
  - Déploiement : Conteneurisation Docker et API (ex: FastAPI).
  - Inférence Hybride : Serving vs Inférence Locale via `DVC checkout`.
  - Monitoring technique (Grafana) et surveillance du Drift (Evidently AI).

- Résultats (Tableaux et Figures)

  - Figures (Captures d'écran) :
    - DAG Apache Airflow réussi.
    - Tableaux de comparaison MLflow UI.
    - Dashboard de Monitoring Grafana.
    - Visualisation du Drift.
  - Tableaux :
    - Comparaison des métriques ML (Précision, Rappel, F1, etc.).

- Conclusion

  - Bilan général du projet.
  - Difficultés techniques rencontrées.

- Références

Date limite : **Mardi 17 Mai 2026 à minuit.**